教程：编写 Hello World Windows 驱动程序（KMDF内核模式驱动程序框架）

2025/02/06

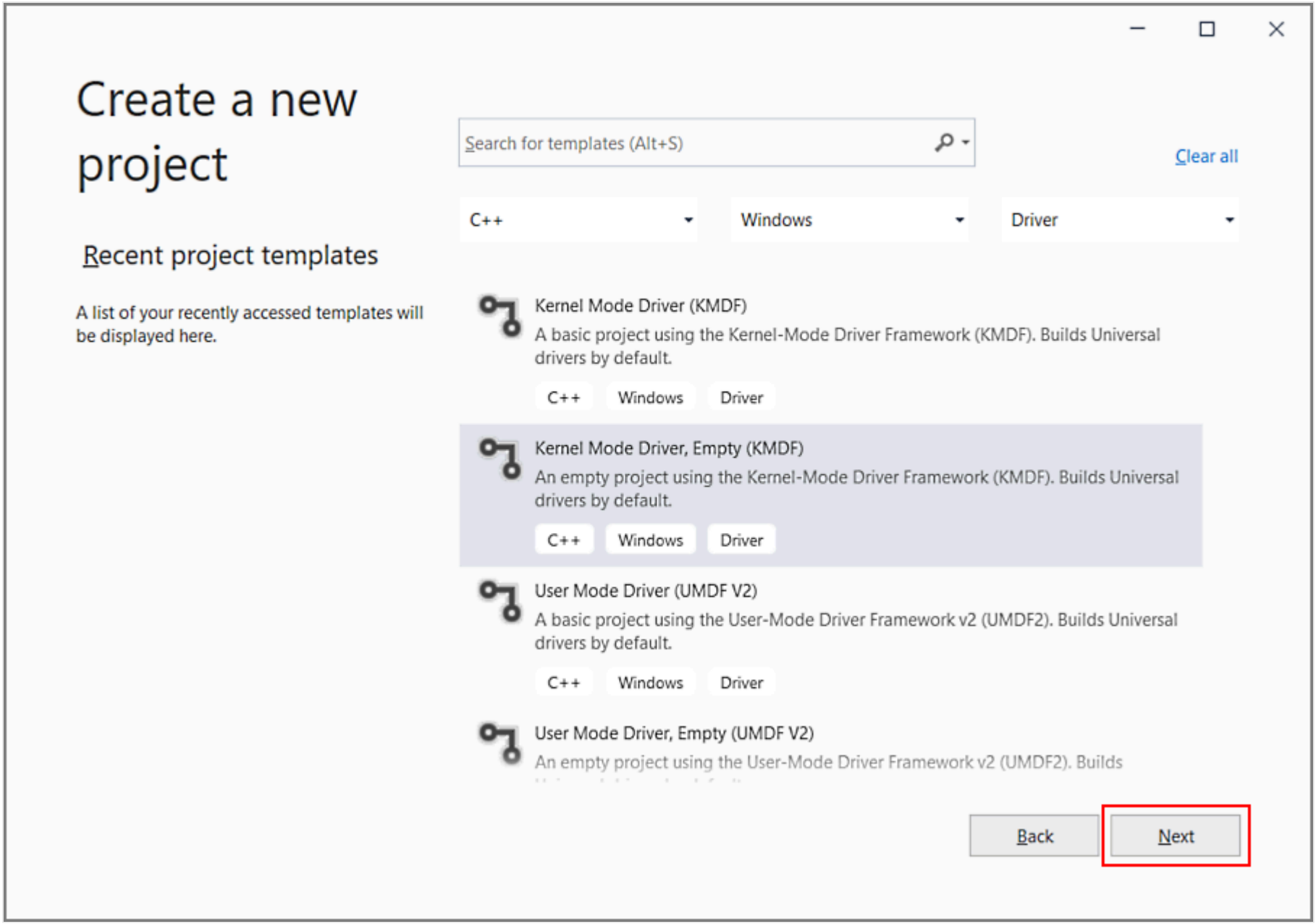
本文介绍如何使用内核模式驱动程序框架 (KMDF) 编写小型通用 Windows 驱动程序，然后在单独的计算机上部署和安装驱动程序。

先决条件

- 按照步骤安装 Windows 驱动程序工具包 (WDK)。 安装 WDK 时，需要包括 Windows 调试工具。
- 安装 Visual Studio 2022。 安装 Visual Studio 2022 时，选择**使用 C++ 进行桌面开发**工作负荷，然后在**单独组件**下添加：
  - MSVC v143 - VS 2022 C++ ARM64/ARM64EC Spectre 缓解库（最新版本）
  - MSVC v143 - VS 2022 C++ x64/x86 Spectre 缓解库（最新版本）
  - 带有 Spectre 缓解库的适用于最新 v143 生成工具的 C++ ATL (ARM64/ARM64EC)
  - 带有 Spectre 缓解库的适用于最新 v143 生成工具的 C++ ATL (x86 & x64)
  - 带有 Spectre 缓解库的适用于最新 v143 生成工具的 C++ MFC (ARM64/ARM64EC)
  - 带有 Spectre 缓解库的适用于最新 v143 生成工具的 C++ MFC (x86 & x64)
  - Windows 驱动程序工具包 (WDK)

创建和生成驱动程序

1. 打开 Microsoft Visual Studio。 在“**文件**”菜单上，选择**新建** > **项目**。
2. 在“新建项目”对话框的左侧下拉列表中，选择“C++”，在中间下拉列表中，选择“Windows”，然后在右侧下拉列表中，选择“驱动程序”。
3. 从项目类型列表中选择“内核模式驱动程序，空(KMDF)”。 选择**下一步**。


Create a new project
Search for templates (Alt+S)
Clear all
C++
Windows
Driver
Recent project templates
A list of your recently accessed templates will be displayed here.
Kernel Mode Driver (KMDF)
A basic project using the Kernel-Mode Driver Framework (KMDF). Builds Universal drivers by default.
C++ Windows Driver
Kernel Mode Driver, Empty (KMDF)
An empty project using the Kernel-Mode Driver Framework (KMDF). Builds Universal drivers by default.
C++ Windows Driver
User Mode Driver (UMDF V2)
A basic project using the User-Mode Driver Framework v2 (UMDF2). Builds Universal drivers by default.
C++ Windows Driver
User Mode Driver, Empty (UMDF V2)
An empty project using the User-Mode Driver Framework v2 (UMDF2). Builds
Back
Next

提示

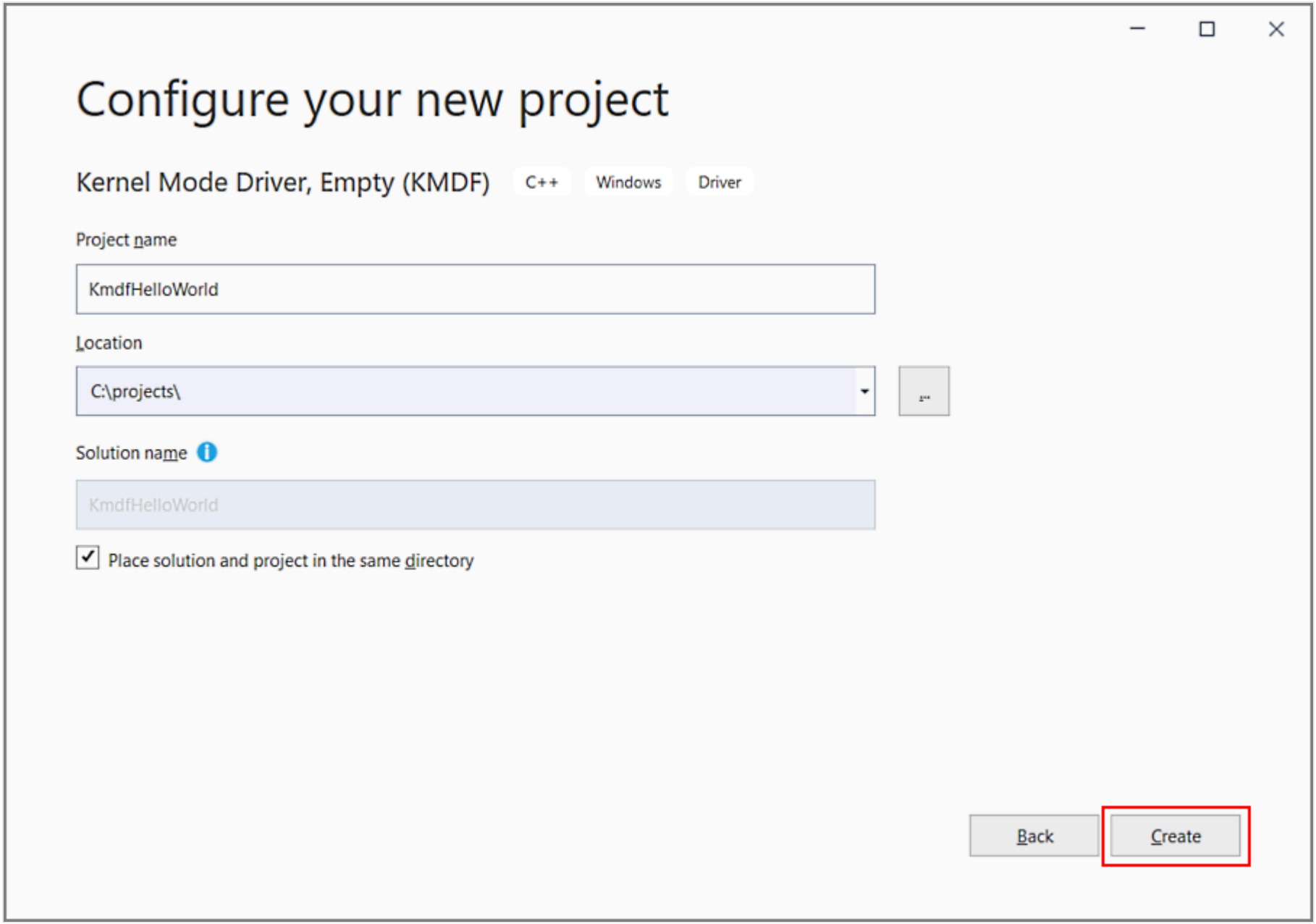
如果在 Visual Studio 中找不到驱动程序项目模板，则表示 WDK Visual Studio 扩展未正确安装。 若要解决此问题，请启动 **Visual Studio Installer**，选择 **修改**，在 **单个组件** 选项卡中添加 **Windows 驱动程序工具包**，然后选择 **修改**。

4. 在“配置新项目”对话框中，在“项目名称”字段中输入“KmdfHelloWorld”。

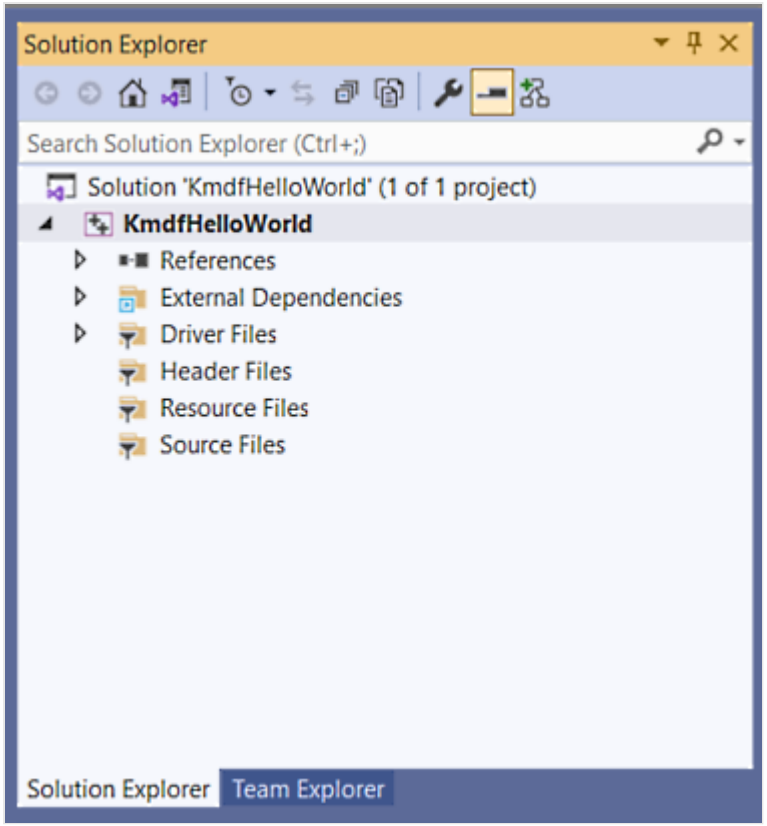
> **备注**
>
> 在创建新的 KMDF 或 UMDF 驱动程序时，必须选择一个不多于 32 个字符的驱动程序名称。 此长度限制在 wdfglobals.h 中定义。

5. 在“位置”字段中，输入要在其中创建新项目的目录。

6. 选中“将解决方案和项目置于同一目录中”，然后选择“创建”。



Visual Studio 将创建一个项目和一个解决方案。 可以在“解决方案资源管理器”窗口中看到它们。 （如果“解决方案资源管理器”窗口不可见，请从**视图**菜单中选择**解决方案资源管理器**。）该解决方案有一个名为 KmdfHelloWorld 的驱动程序项目。



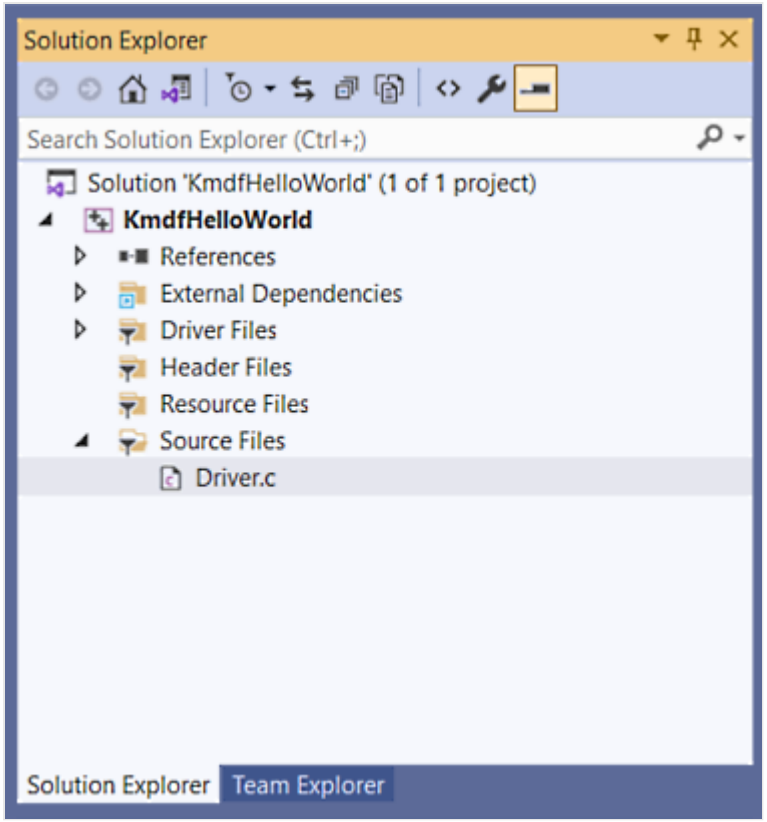
7. 在 **解决方案资源管理器** 窗口中，右键单击 **解决方案**“KmdfHelloWorld”（**共 1 个项目**） 并选择 Configuration Manager。 为驱动程序项目选择配置和平台。 例如，选择“调试”和“x64”。

8. 在 **解决方案资源管理器** 窗口中，右键单击 KmdfHelloWorld 项目，选择 **添加**，然后选择 **新项**。

9. 在“**添加新项** 对话框中，输入”Driver.c”。

> ⓘ **备注**
>
> 文件扩展名为 .c，不是 .cpp。

选择“” **添加** “”。 Driver.c 文件添加在源文件下，如下所示。



编写第一个驱动程序代码

创建空的 Hello World 项目并添加了 Driver.c 源文件后，即可通过实现两个基本事件回调函数编写驱动程序运行所需的最基本的代码。

1. 在 Driver.c 中，首先包括以下头文件：

```
C

#include <ntddk.h>
```

```
#include <wdf.h>
```

> **提示**
>
> 如果无法添加 `Ntddk.h`，请打开“配置”->“C/C++”->“常规”->“其他包含目录”并添加，将 `C:\Program Files (x86)\Windows Kits\10\Include\<build#>\km` 替换为 WDK 安装中的相应目录 `<build#>`。

Ntddk.h 包含所有驱动程序的核心 Windows 内核定义，而 Wdf.h 包含基于 Windows 驱动程序框架 (WDF) 的驱动程序的定义。

2. 接下来，为两个回调提供声明：

```c
DRIVER_INITIALIZE DriverEntry;
EVT_WDF_DRIVER_DEVICE_ADD KmdfHelloWorldEvtDeviceAdd;
```

3. 使用以下代码编写 *DriverEntry*：

```c
NTSTATUS
DriverEntry(
    _In_ PDRIVER_OBJECT     DriverObject,
    _In_ PUNICODE_STRING    RegistryPath
)
{
    // NTSTATUS variable to record success or failure
    NTSTATUS status = STATUS_SUCCESS;

    // Allocate the driver configuration object
    WDF_DRIVER_CONFIG config;
```

```
    // Print "Hello World" for DriverEntry
    KdPrintEx(( DPFLTR_IHVDRIVER_ID, DPFLTR_INFO_LEVEL, "KmdfHelloWorld: DriverEntry\n" ));

    // Initialize the driver configuration object to register the
    // entry point for the EvtDeviceAdd callback, KmdfHelloWorldEvtDeviceAdd
    WDF_DRIVER_CONFIG_INIT(&config,
                           KmdfHelloWorldEvtDeviceAdd
                           );

    // Finally, create the driver object
    status = WdfDriverCreate(DriverObject,
                             RegistryPath,
                             WDF_NO_OBJECT_ATTRIBUTES,
                             &config,
                             WDF_NO_HANDLE
                             );
    return status;
}
```

DriverEntry 是所有驱动程序的入口点，就像 `Main()` 适用于许多用户模式应用程序一样。 *DriverEntry* 的任务是初始化驱动程序范围的结构和资源。 在此示例中，你在 *DriverEntry*中打印了“Hello World”，将驱动程序对象配置为注册 *EvtDeviceAdd* 回调的入口点，接着创建驱动程序对象并返回。

驱动程序对象充当你可能在驱动程序中创建的所有其他框架对象的父对象，这些框架对象包括设备对象、I/O 队列、计时器、旋转锁等。有关框架对象的详细信息，请参阅框架对象简介。

> **提示**
>
> 对于 *DriverEntry*，我们强烈建议将名称保留为“DriverEntry”，以便进行代码分析和调试。

4. 接下来，使用以下代码编写 *KmdfHelloWorldEvtDeviceAdd*：

```c
NTSTATUS
KmdfHelloWorldEvtDeviceAdd(
    _In_    WDFDRIVER       Driver,
    _Inout_ PWDFDEVICE_INIT DeviceInit
)
{
    // We're not using the driver object,
    // so we need to mark it as unreferenced
    UNREFERENCED_PARAMETER(Driver);

    NTSTATUS status;

    // Allocate the device object
    WDFDEVICE hDevice;

    // Print "Hello World"
    KdPrintEx(( DPFLTR_IHVDRIVER_ID, DPFLTR_INFO_LEVEL, "KmdfHelloWorld: KmdfHelloWorldEvtDeviceAdd\n" ));

    // Create the device object
    status = WdfDeviceCreate(&DeviceInit,
                             WDF_NO_OBJECT_ATTRIBUTES,
                             &hDevice
                             );
    return status;
}
```

当系统检测到设备到达时，系统会调用 *EvtDeviceAdd*。 它的任务是初始化该设备的结构和资源。 在此示例中，你输出了 *EvtDeviceAdd*的“Hello World”消息，创建了设备对象并返回。 在你编写的其他驱动程序中，可以为硬件创建 I/O 队列，为特定于设备的信息设置设备上下文存储空间，或执行准备设备所需的其他任务。

提示

对于设备添加回调，请注意以驱动程序名称为前缀对回调命名的方式 (*KmdfHelloWorld*EvtDeviceAdd)。 通常，我们建议以这种方式命名驱动程序功能，以区别于其他驱动程序的功能。 *DriverEntry* 是完全应该这样命名的唯一一项。

5. 您的完整 Driver.c 现在如下所示：

C

```c
#include <ntddk.h>
#include <wdf.h>
DRIVER_INITIALIZE DriverEntry;
EVT_WDF_DRIVER_DEVICE_ADD KmdfHelloWorldEvtDeviceAdd;

NTSTATUS
DriverEntry(
    _In_ PDRIVER_OBJECT     DriverObject,
    _In_ PUNICODE_STRING    RegistryPath
)
{
    // NTSTATUS variable to record success or failure
    NTSTATUS status = STATUS_SUCCESS;

    // Allocate the driver configuration object
    WDF_DRIVER_CONFIG config;

    // Print "Hello World" for DriverEntry
    KdPrintEx(( DPFLTR_IHVDRIVER_ID, DPFLTR_INFO_LEVEL, "KmdfHelloWorld: DriverEntry\n" ));

    // Initialize the driver configuration object to register the
    // entry point for the EvtDeviceAdd callback, KmdfHelloWorldEvtDeviceAdd
    WDF_DRIVER_CONFIG_INIT(&config,
                           KmdfHelloWorldEvtDeviceAdd
                           );

    // Finally, create the driver object
    status = WdfDriverCreate(DriverObject,
```

```
                            RegistryPath,
                            WDF_NO_OBJECT_ATTRIBUTES,
                            &config,
                            WDF_NO_HANDLE
                            );
    return status;
}

NTSTATUS
KmdfHelloWorldEvtDeviceAdd(
    _In_    WDFDRIVER       Driver,
    _Inout_ PWDFDEVICE_INIT DeviceInit
)
{
    // We're not using the driver object,
    // so we need to mark it as unreferenced
    UNREFERENCED_PARAMETER(Driver);

    NTSTATUS status;

    // Allocate the device object
    WDFDEVICE hDevice;

    // Print "Hello World"
    KdPrintEx(( DPFLTR_IHVDRIVER_ID, DPFLTR_INFO_LEVEL, "KmdfHelloWorld: KmdfHelloWorldEvtDeviceAdd\n" ));

    // Create the device object
    status = WdfDeviceCreate(&DeviceInit,
                            WDF_NO_OBJECT_ATTRIBUTES,
                            &hDevice
                            );
    return status;
}
```

6. 保存 Driver.c。

此示例演示了驱动程序的基本概念：它们是一个“回调集合”，一旦初始化，就处于待命状态并等待系统在需要时调用。 系统调用可以是新的设备到达事件、来自用户模式应用程序的 I/O 请求、系统电源关闭事件、来自另一个驱动程序的请求，或者是用户意外拔下设备时的意外移除事件。 幸运的是，就“Hello World”而言，只需操心驱动程序和设备的创建。

接下来，生成驱动程序。

生成驱动程序

1. 在 **解决方案资源管理器** 窗口中，右键单击 **解决方案**“KmdfHelloWorld”（**共 1 个项目**） 并选择 **Configuration Manager**。 为驱动程序项目选择配置和平台。 对于本练习，请选择**调试**和 **x64**。

2. 在 **解决方案资源管理器** 窗口中，右键单击 **KmdfHelloWorld**，然后选择 **属性**。 在“Wpp 跟踪”>“所有选项”中，将“运行 Wpp 跟踪”设置为“否”。 依次选择“应用”、“确定” 。

3. 若要生成驱动程序，请从“生成”菜单中选择“生成解决方案”。 Visual Studio 在“输出”窗口中显示生成进度。 （如果 **输出** 窗口不可见，请从 **视图** 菜单中选择 **输出**。验证解决方案是否已成功生成时，可以关闭 Visual Studio。

4. 若要查看生成的驱动程序，请在文件资源管理器中转到 **KmdfHelloWorld** 文件夹，然后查看 **x64\Debug\KmdfHelloWorld**。 该文件夹包括：

   - KmdfHelloWorld.sys - 内核模式驱动程序文件
   - KmdfHelloWorld.inf - 在安装驱动程序时 Windows 使用的信息文件
   - KmdfHelloWorld.cat - 安装程序验证驱动程序的测试签名所使用的目录文件

> **提示**
>
> 如果在生成驱动程序时看到 `DriverVer set to a date in the future`，则请更改驱动程序项目设置，让 Inf2Cat 设置 `/uselocaltime`。 为此，请使用 **配置属性** ->**Inf2Cat** ->**常规** ->**使用本地时间**。 现在，Stampinf 和 Inf2Cat 都使用本地时间。

部署驱动程序

通常，在测试和调试驱动程序时，调试程序和驱动程序会在不同的计算机上运行。 运行调试程序的计算机称为“主计算机”，运行驱动程序的计算机称为“目标计算机”。 目标计算机也称为“测试计算机”。

到目前为止，你已使用 Visual Studio 在主计算机上生成驱动程序。 现在，你需要配置目标计算机。

1. 按照预配计算机以便进行驱动程序部署和测试 (WDK 10) 中的说明进行操作。

   > **提示**
   >
   > 按照步骤使用网络电缆自动预配目标计算机时，请记下端口和密钥。 以后，你将在调试步骤中使用它们。 在此示例中，使用 **50000** 作为端口，**1.2.3.4** 作为密钥。
   >
   > 在实际的驱动程序调试方案中，我们建议使用 KDNET 生成的密钥。 有关如何使用 KDNET 生成一个随机密钥的详细信息，请参阅**调试驱动程序 - 分步实验室（Sysvad 内核模式）**主题。

2. 在主计算机上，在 Visual Studio 中打开你的解决方案。 可以在 KmdfHelloWorld 文件夹中双击解决方案文件 KmdfHelloWorld.sln。

3. 在 **解决方案资源管理器** 窗口中，右键单击 **KmdfHelloWorld** 项目，然后选择 **属性**。

4. 转到“驱动程序安装”>“部署”。

5. 对于“目标设备名称”，请选择配置用于测试和调试的计算机名。 在本练习中，我们使用名为 MyTestComputer 的计算机。

6. 若要确保测试最新版本的驱动程序，请在部署 **之前检查**删除以前的驱动程序版本。

7. 选择“硬件 ID 驱动程序更新”，然后输入驱动程序的硬件 ID。 在本练习中，硬件 ID 为 Root\KmdfHelloWorld。 选择“确定”。

   > **备注**

在本练习中，硬件 ID 并不用于标识真实的硬件。 它标识了虚构设备，该设备位于**设备树**中，作为根节点的子节点。 对于真实的硬件，不要选择“硬件 ID 驱动程序更新”，而要选择“安装并验证”。 可在驱动程序的信息（INF）文件中看到硬件 ID。 在“解决方案资源管理器”窗口中，转到“KmdfHelloWorld”>“驱动程序文件”，然后双击 KmdfHelloWorld.inf。 硬件 ID 位于 [Standard.NT$ARCH$] 下。

```
C++

[Standard.NT$ARCH$]
%KmdfHelloWorld.DeviceDesc%=KmdfHelloWorld_Device, Root\KmdfHelloWorld
```

8. 在“生成”菜单上，选择“部署解决方案”。 Visual Studio 会自动将安装和运行驱动程序所需的文件复制到目标计算机。 部署可能会需要一两分钟的时间。

   部署驱动程序时，驱动程序文件将复制到测试计算机上的 %Systemdrive%\drivertest\drivers 文件夹。 如果部署期间发生错误，可以查看这些文件是否已复制到测试计算机。 请确认 .inf、.cat、测试证书和 .sys 文件以及其他任何必要的文件均位于 %systemdrive%\drivertest\drivers 文件夹中。

   有关部署驱动程序的详细信息，请参阅将驱动程序部署到测试计算机。

安装驱动程序

将 Hello World 驱动程序部署到目标计算机后，现在安装驱动程序。 如果你之前使用“自动”选项通过 Visual Studio 预配了目标计算机，则在预配过程中，Visual Studio 会将目标计算机设置为运行测试签名驱动程序。 现在，你只需使用 DevCon 工具安装驱动程序即可。

1. 在主计算机上，导航到 WDK 安装中的“Tools”文件夹，然后找到 DevCon 工具。 例如，在以下文件夹中查看：

   C:\Program Files (x86)\Windows Kits\10\Tools\x64\devcon.exe

   将 DevCon 工具复制到远程计算机。

2. 在目标计算机上，导航到包含驱动程序文件的文件夹，然后运行 DevCon 工具，以安装驱动程序。

   a. devcon 工具的一般语法如下，用于安装驱动程序：

   devcon 安装 <INF 文件><硬件 ID>

   安装此驱动程序所需的 INF 文件是 KmdfHelloWorld.inf。 INF 文件包含用于安装驱动程序二进制文件 *KmdfHelloWorld.sys* 的硬件 ID。请注意，位于 INF 文件中的硬件 ID 是 **Root\KmdfHelloWorld**。

   b. 以管理员身份打开命令提示符窗口。 导航到内置驱动程序 .sys 文件所在的文件夹，然后输入以下命令：

   devcon 安装 kmdfhelloworld.inf root\kmdfhelloworld

   如果收到一条关于 devcon 未被识别的错误消息，请尝试添加 devcon 工具的路径。 例如，如果已将其复制到目标计算机上名为 C:\Tools 的文件夹，则请尝试使用以下命令：

   c:\tools\devcon install kmdfhelloworld.inf root\kmdfhelloworld

   此时会显示一个对话框，指示测试驱动程序是未签名的驱动程序。 选择“仍然安装此驱动程序”以继续。

调试驱动程序

在目标计算机上安装 KmdfHelloWorld 驱动程序后，可以从主计算机远程附加调试器。

1. 在主计算机上，以管理员身份打开命令提示符窗口。 切换到 WinDbg.exe 目录。 你将使用安装 Windows 工具包过程中安装的 Windows 驱动程序工具包 (WDK) 中的 x64 版本 WinDbg.exe。 下面是 WinDbg.exe 的默认路径：

   C:\Program Files (x86)\Windows Kits\10\Debuggers\x64

2. 使用以下命令启动 WinDbg 以连接到目标计算机上的内核调试会话。 端口和密钥的值应该与你预配目标计算机所使用的值相同。 你在部署步骤中使用的值是，端口用 **50000**，键值用 **1.2.3.4**。 *K* 标志指示这是内核调试会话。

   WinDbg -k net:port=50000,key=1.2.3.4

3. 在“调试”菜单上，选择“中断”。 主计算机上的调试器闯入目标计算机。 在“调试程序命令”窗口中，你可以看到内核调试命令提示符：**kd>**。

4. 此时，可以试验调试程序，方法是在 **kd>** 提示符处输入命令。 例如，可以尝试使用以下命令：

   - lm
   - .sympath
   - .reload
   - x KmdfHelloWorld!*

5. 若要让目标计算机再次运行，请从“调试”菜单中选择“执行”，或者按“g”，然后按“Enter”。

6. 若要停止调试会话，请从**调试**菜单中选择**分离调试器**。

   > ⓘ **重要**

请确保在退出调试程序之前使用“执行”命令让目标计算机再次运行，否则目标计算机将仍然对你的鼠标和键盘输入无响应，因为它仍在与调试程序通话。

有关驱动程序调试过程的详细分步演练，请参阅调试通用驱动程序 - 分步实验室（回显内核模式）。

有关远程调试的详细信息，请参阅使用 WinDbg 远程调试。

相关文章

Windows 调试工具

调试通用驱动程序 - 分步骤实验(Echo Kernel-Mode)

编写你的第一个驱动程序